

AMATH 483 / 583 (Roche) - Homework Set 2

Due Wednesday April 16, 5pm PT

April 10, 2025

Homework 2 (82 points)

1. (+10) Write a C++ program that finds a practical measure of your machine's SP (32 bit) and DP (64 bit) floating point precision by taking the difference of 2 numbers and comparing these to zero in the same precision. What value do you obtain for $\epsilon_{\text{machine}}$ in both precisions? Hint: A loop (j) will be helpful here $1 - (1 + \frac{1}{2^j})$ should do it.

```
1  #include <iostream>
2  #include <iomanip>
3  #include <cmath>
4  #include <limits>
5
6  int main() {
7      // Problem 1
8      float epsSP = 0.0f;
9      int j = 1;
10     float diffSP = 0.0f;
11     while (true) {float term = 1.0f / powf(2.0f, j); // Compute 1/2^j using powf
12         // Compute the difference: (1 + 1/2^j) - 1
13         diffSP = (1.0f + term) - 1.0f;
14         // If the difference is 0, then term is too small to be distinguished
15         if (diffSP == 0.0f) {epsSP = 1.0f / powf(2.0f, j - 1);
16             break;}
17         // Otherwise, increment j and repeat.
18         j++;}
19
20     double epsDP = 0.0;
21     j = 1; // Reset counter for double computation
22     double diffDP = 0.0;
23     while (true) {
24         // Compute 1/2^j
25         double term = 1.0 / pow(2.0, j);
26         // Compute the difference: (1 + 1/2^j) - 1
27         diffDP = (1.0 + term) - 1.0;
28         // When the term is too small, diffDP will be 0.
29         if (diffDP == 0.0) {epsDP = 1.0 / pow(2.0, j - 1);
30             break;}
31         j++;}
32
33     std::cout << std::scientific << std::setprecision(6);
34     std::cout << "Problem 1 Solution: "<< std::endl;
35     std::cout << "Practical measure of this machine's SP (32 bit) = "
36     |<< std::numeric_limits<float>::epsilon() << std::endl;
37     std::cout << "Practical measure of this machine's DP (64 bit) = "
38     |<< std::numeric_limits<double>::epsilon() << std::endl;
39     return 0;
40 }
```

Problem 1 Solution:

Practical measure of this machine's SP (32 bit) = 1.192093e-07

Practical measure of this machine's DP (64 bit) = 2.220446e-16

2. (+12) What are the largest and smallest SP (32 bit) and DP (64 bit) numbers that can be represented in IEEE floating point representation? Show work in terms of sign, mantissa, and exponent.

1) Largest SP number in IEEE floating point representation

normalized :

Sign bit, $S = 0$ (to get largest positive)

exponent bit, $e = 254$ (00000001 to 11111110) that corresponds to the exponent $E = e - \text{bias} = 254 - (2^{8-1} - 1) = 127$

fraction bit, $f = 2^{-1} + 2^{-2} + 2^{-3} + \dots + 2^{-23}$

mantissa: $= 1 - 2^{-23}$

$$\begin{aligned} M &= 1 + f \\ &= 1 + 1 - 2^{-23} \\ &= 2 - 2^{-23} \end{aligned}$$

$$\begin{aligned} V &= (-1)^S \cdot M \cdot 2^E \\ &= -1^0 \cdot (2 - 2^{-23}) \cdot 2^{127} \\ &= 1 \cdot (2 - 2^{-23}) \cdot 2^{127} \\ &= 3.40282347 \cdot 10^{38} \end{aligned}$$

denormalized :

$$\begin{aligned} M &= f = 1 - 2^{-n} \\ &= 1 - 2^{-23} \end{aligned}$$

$$E = -2^{K-1} + 2 = -126$$

$$\begin{aligned} V &= (1 - 2^{-23}) \cdot 2^{-126} \\ &= 1.17549421 \cdot 10^{-38} \end{aligned}$$

$$3.40282347 \cdot 10^{38}$$

2) Smallest SP number in IEEE floating point representation

To get smallest, have sign bit = 1

$$\begin{aligned} V &= (-1) \cdot (2 - 2^{-23}) \cdot 2^{127} \\ &= -3.40282347 \cdot 10^{38} \end{aligned}$$

$$-3.40282347 \cdot 10^{38}$$

3) Largest DP number in IEEE floating point representation

normalized :

sign bit, $s = 0$ (to get largest positive)

exponent bit, $e = 2046$ (0000000001 to 1111111110) that corresponds to the exponent $E = e - \text{bias} = 2046 - (2^{11-1} - 1) = 1023$

fraction bit, $f = 2^{-1} + 2^{-2} + 2^{-3} + \dots + 2^{-52}$
 $= 1 - 2^{-52}$

mantissa :

$$\begin{aligned} M &= 1 + f \\ &= 1 + 1 - 2^{-52} \\ &= 2 - 2^{-52} \end{aligned}$$

$$\begin{aligned} V &= (-1)^s \cdot M \cdot 2^E \\ &= -1^0 \cdot (2 - 2^{-52}) \cdot 2^{1023} \\ &= 1.79769313 \cdot 10^{308} \end{aligned}$$

denormalized :

$$\begin{aligned} M &= f = 1 - 2^{-n} \\ &= 1 - 2^{-52} \\ E &= -2^{K-1} + 2 = -1022 \\ V &= (1 - 2^{-52}) \cdot 2^{-1022} \\ &= 2^{-1022} \\ &= 2.22507386 \cdot 10^{-308} \end{aligned}$$

$1.79769313 \cdot 10^{308}$

4) Smallest DP number in IEEE floating point representation

To get smallest, have sign bit = 1

$$\begin{aligned} V &= (-1) \cdot (2 - 2^{-52}) \cdot 2^{1023} \\ &= -1.79769313 \cdot 10^{308} \end{aligned}$$

$-1.79769313 \cdot 10^{308}$

3. (+5) Write a C++ program to multiply the integers $200 * 300 * 400 * 500$ on your computer? What is the result? Name the effect you observe.

```
1  #include <iostream>
2  #include <iomanip>
3  #include <cmath>
4  #include <limits>
5
6  int main() {
7
8      // Problem 3
9      int a = 200, b = 300, c = 400, d = 500;
10     int product_pbm3 = a * b * c * d;
11     std::cout << "Problem 3 Solution: " << std::endl;
12     std::cout << "Multiply the integers, 200 * 300 * 400 * 500 = " << product_pbm3 << std::endl;
13
14     return 0;
15 }
```

Problem 3 Solution:

Multiply the integers, $200 * 300 * 400 * 500 = -884901888$

The result of multiplying the integers, $200 * 300 * 400 * 500$ is -884901888. This result is different to the mathematical multiplication which is 12,000,000,000. The reason is because 12,000,000,000 is greater than the maximum that 32-bit signed integer range which is $[2^{(-31)}, 2^{31-1}]$. In these cases, C++ wraps around modulo 2^{32} which is called as integer overflow. So C++ computes the multiplication by $12,000,000,000 \bmod 2^{32}$. $2^{32} = 4,294,967,296$ and as 4,294,967,296 goes twice into 12,000,000,000. Finding the remainder of this, $12,000,000,000 - (4,294,967,296 * 2) = 3,410,065,408$.

Any 32-bit value above 2^{31-1} has the sign bit set to 1, which would appear negative. So to get the signed value, it would be $\text{signedvalue} = \text{unsignedvalue} - 2^{32} = 3,410,065,408 - 4,294,967,296 = -884,901,888$. Therefore, C++ gets the unsigned results of $12,000,000,000 \bmod 2^{32} = 3,410,065,408$, but as "int" is signed, the computer sees the patterns as 32-bit pattern which is $3,410,065,408 - 4,294,967,296 = -884,901,888$.

The effect I observe is the "Integer Overflow."

4. (+5) Given C++ code segment below, what is the final value of *counter*?

```
unsigned int counter = 0;  
for (int i = 0; i < 3; ++i) --counter;
```

```
1  #include <iostream>  
2  #include <iomanip>  
3  #include <cmath>  
4  #include <limits>  
5  
6  int main() {  
7  
8      // Problem 4  
9      unsigned int counter = 0;  
10     for (int i = 0; i < 3; ++i)  
11         -- counter;  
12  
13     std::cout << "Problem 4 Solution: " << std::endl;  
14     std::cout << "counter = " << counter << std::endl;  
15  
16     return 0;  
17 }
```

Problem 4 Solution:
counter = 4294967293

counter is initially : counter = 0

first -- counter : counter = $2^{32} - 1 = 4294967295$

second -- counter : counter = $4294967295 - 1 = 4294967294$

third -- counter : counter = $4294967294 - 1 = 4294967293$

4294967293

5. (+10) Count and report how many IEEE SP (32 bit) normalized and denormalized floating point numbers there are. Please count and label infinities and NaNs as well. Show work.

exponent bits	decimal value	meaning
00000000	0	denormalized numbers or zero
00000001 - 11111110	1 - 254	normalized numbers
11111111	255	infinity or NaN

IEEE SP (32 bit) normlized floating point numbers

$$\begin{aligned}
 \text{answer} &= \text{possible exponent values} \cdot \text{fraction field} \cdot \text{possibilities of signs} \\
 &= 254 \cdot 2^{23} \cdot 2 \\
 &= 4261412864
 \end{aligned}$$

4261412864

IEEE SP (32 bit) denormlized floating point numbers

$$\begin{aligned}
 \text{answer} &= \text{possible exponent values} \cdot \text{fraction field} \cdot \text{possibilities of signs} \\
 &= 1 \cdot (2^{23} - 1) \cdot 2 \\
 &= 16777214
 \end{aligned}$$

16777214

IEEE SP (32 bit) infinite floating point numbers

$$\begin{aligned}
 \text{answer} &= \text{possible exponent values} \cdot \text{fraction field} \cdot \text{possibilities of signs} \\
 &= 1 \cdot 1 \cdot 2 \\
 &= 2
 \end{aligned}$$

2

IEEE SP (32 bit) NaN floating point numbers

$$\begin{aligned}
 \text{answer} &= \text{possible exponent values} \cdot \text{fraction field} \cdot \text{possibilities of signs} \\
 &= 1 \cdot (2^{23} - 1) \cdot 2 \\
 &= 16777214
 \end{aligned}$$

16777214

6. (+15) Consider a 6 bit floating point system with $s = 1$ (1 sign bit), $k = 3$ (3 bit exponent field), and $n = 2$ (2 bit mantissa).

- Calculate by hand all the representable normalized numbers. Show work.
- Calculate by hand all the representable denormalized numbers. Show work.
- Plot both sets of numbers (ignoring NaNs and infinities) as a number line to see the gaps of (un)representable numbers.

a)

$$\text{bias} = 2^{k-1} - 1 = 3$$

$$E = e - \text{bias}$$

$$= (e_2 e_1 e_0) - 3$$

$$M = 1 + f$$

$$f \sim f_n, f_{n-1}$$

$$\{0, 1\} \cdot \frac{1}{2^1} \quad \{0, 1\} \cdot \frac{1}{2^2}$$

normalized when exp has not $e_i = 0 \forall i$ and not $e_i = 1 \forall i$

e_2	e_1	e_0	e	$E = e - \text{bias}$	n_1	n_0	f	$M = 1 + f$	$V = M \cdot 2^E$
0	0	1	1	-2	0	0	0	1	$1 \cdot 2^{-2} = 0.25$
0	0	1	1	-2	0	1	$\frac{1}{2^2} = \frac{1}{4}$	1.25	$1.25 \cdot 2^{-2} = 0.3125$
0	0	1	1	-2	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^{-2} = 0.375$
0	0	1	1	-2	1	1	$1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} = \frac{3}{4}$	1.75	$1.75 \cdot 2^{-2} = 0.4375$
0	1	0	2	-1	0	0	0	1	$1 \cdot 2^{-1} = 0.5$
0	1	0	2	-1	0	1	$\frac{1}{4}$	1.25	$1.25 \cdot 2^{-1} = 0.625$
0	1	0	2	-1	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^{-1} = 0.75$
0	1	0	2	-1	1	1	$\frac{3}{4}$	1.75	$1.75 \cdot 2^{-1} = 0.875$
0	1	1	3	0	0	0	0	1	$1 \cdot 2^0 = 1$
0	1	1	3	0	0	1	$\frac{1}{4}$	1.25	$1.25 \cdot 2^0 = 1.25$
0	1	1	3	0	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^0 = 1.5$
0	1	1	3	0	1	1	$\frac{3}{4}$	1.75	$1.75 \cdot 2^0 = 1.75$
1	0	0	4	1	0	0	0	1	$1 \cdot 2^1 = 2$
1	0	0	4	1	0	1	$\frac{1}{4}$	1.25	$1.25 \cdot 2^1 = 2.5$
1	0	0	4	1	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^1 = 3$
1	0	0	4	1	1	1	$\frac{3}{4}$	1.75	$1.75 \cdot 2^1 = 3.5$
1	0	1	5	2	0	0	0	1	$1 \cdot 2^2 = 4$
1	0	1	5	2	0	1	$\frac{1}{4}$	1.25	$1.25 \cdot 2^2 = 5$
1	0	1	5	2	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^2 = 6$
1	0	1	5	2	1	1	$\frac{3}{4}$	1.75	$1.75 \cdot 2^2 = 7$
1	1	0	6	3	0	0	0	1	$1 \cdot 2^3 = 8$
1	1	0	6	3	0	1	$\frac{1}{4}$	1.25	$1.25 \cdot 2^3 = 10$
1	1	0	6	3	1	0	$\frac{1}{2}$	1.5	$1.5 \cdot 2^3 = 12$
1	1	0	6	3	1	1	$\frac{3}{4}$	1.75	$1.75 \cdot 2^3 = 14$

$\pm 0.25, \pm 0.3125, \pm 0.375, \pm 0.4375, \pm 0.5, \pm 0.625, \pm 0.75, \pm 0.875, \pm 1, \pm 1.25, \pm 1.5, \pm 1.75, \pm 2, \pm 2.5, \pm 3, \pm 3.5, \pm 4, \pm 5, \pm 6, \pm 7, \pm 8, \pm 10, \pm 12, \pm 14$

b)

denormalized when $e_i = 0 \forall i$

e_2	e_1	e_0	e	$E = 1 - \text{bias}$	n_1	n_0	f	$M = f$	$V = M \cdot 2^E$
0	0	0	0	-2	0	0	0	0	0
0	0	0	0	-2	0	1	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4} \cdot 2^{-2} = 0.0625$
0	0	0	0	-2	1	0	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2} \cdot 2^{-2} = 0.125$
0	0	0	0	-2	1	1	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4} \cdot 2^{-2} = 0.1875$

3 (not including 0)

→ 3

but each can also have sign as negative so it will be $3 \cdot 2 = 6$

$$3 \cdot 2 = 6$$

c)

```
import matplotlib.pyplot as plt

bias = 3 # for 3-bit exponent

# Collect DENORMALIZED values
denorm_vals = []
for s in (0, 1): # sign bit
    sign = +1 if s == 0 else -1
    for f in range(4): # fraction bits
        value = sign * (f / 16)
        denorm_vals.append(value)

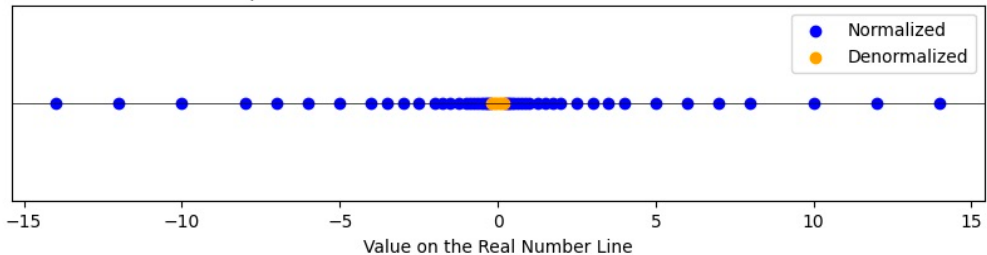
# Collect NORMALIZED values
normal_vals = []
for s in (0, 1):
    sign = +1 if s == 0 else -1
    for E in range(1, 7):
        e = E - bias
        for f in range(4): # fraction bits
            mantissa = 1 + f/4
            value = sign * mantissa * (2 ** e)
            normal_vals.append(value)

denorm_vals = sorted(set(denorm_vals))
normal_vals = sorted(set(normal_vals))

plt.figure(figsize=(10, 2))
plt.scatter(normal_vals, [0]*len(normal_vals),
            color='blue', markers='o', label='Normalized')
plt.scatter(denorm_vals, [0]*len(denorm_vals),
            color='orange', markers='o', label='Denormalized')
plt.legend()
plt.axhline(0, color='black', linewidth=0.5) # baseline
plt.title("Representable Denormalized & Normalized numbers")
plt.xlabel("Value on the Real Number Line")
plt.yticks([1])
```

$\pm 0.0625, \pm 0.125, \pm 0.1875$

Representable Denormalized & Normalized numbers



7. (+10) Conversions. Show work.

(a) Write $(D3B701)_{16}$ as an integer in base-10.

(b) Write $(1010000100111111)_2$ as an integer in base-16, i.e. as a hexadecimal number.

a)

$$(D3B701)_{16} = D \cdot 16^5 + 3 \cdot 16^4 + B \cdot 16^3 + 7 \cdot 16^2 + 0 \cdot 16^1 + 1 \cdot 16^0$$

In decimal, the hexadecimal letters are $A=10$, $B=11$, $C=12$, $D=13$.

$$\begin{aligned}(D3B701)_{16} &= 13 \cdot 16^5 + 3 \cdot 16^4 + 11 \cdot 16^3 + 7 \cdot 16^2 + 0 \cdot 16^1 + 1 \cdot 16^0 \\ &= 13874945\end{aligned}$$

$$(D3B701)_{16} = 13874945_{10}$$

b)

$$(1010000100111111)_2$$

$$1010 \quad 0001 \quad 0011 \quad 1111$$

for each group of 4 bits

$$b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

In hexadecimal, $A=10$ $B=11$ $C=12$ $D=13$ $E=14$ $F=15$

$$1010_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 10_{10}$$

converting decimal 10 to hexadecimal

$$1010_2 = 10_{10} = A_{16}$$

$$0001_2 = 0 + 0 + 0 + 1 \cdot 2^0 = 1_{10}$$

$$0001_2 = 1_{10} = 1_{16}$$

$$0011_2 = 0 + 0 + 1 \cdot 2^1 + 1 \cdot 2^0 = 3_{10}$$

$$0011_2 = 3_{10} = 3_{16}$$

$$1111_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 15_{10}$$

$$1111_2 = 15_{10} = F_{16}$$

$$(1010000100111111)_2 = (A13F)_{16}$$

8. (+5) Are there $a, b, c \in \mathbb{Z}$ s.t. $6a + 9b + 15c = 107$? Show work.

$$\gcd(6, 9, 15) = 3$$

This means $6a + 9b + 15c$ should be a number that is the multiplication of 3.

$$107 \div 3 = 35 \dots \text{remainder } 2$$

This shows 107 is not the multiplication of 3 which tells there are

no $a, b, c \in \mathbb{Z}$ such that $6a + 9b + 15c = 107$.

9. (+10) Equivalence classes modulo n . $\forall a, b \in \mathbf{Z}$ then $a \equiv b \pmod{n}$ means $n \mid (a - b)$ or $a = b + k \cdot n$ and $k \in \mathbf{Z}$. $\mathbf{Z}_n$ is the set of equivalence classes $\{[0], [1], \dots, [n-1]\}$. Is $(\mathbf{Z}_n, +, \cdot)$ a ring? Hint: If $s \in [i]$, then $n \mid (s - i)$. Show work (use ring properties).

addition: $[a] + [b] = [a+b]$

multiplication: $[a] \cdot [b] = [a \cdot b]$

Show that if $[a] = [a']$ and $[b] = [b']$ then,

1. $[a+b] = [a'+b']$
2. $[a \cdot b] = [a' \cdot b']$

If $a \equiv a' \pmod{n}$ $b \equiv b' \pmod{n}$ then, $n \mid (a - a')$ and $n \mid (b - b')$

1. Show $[a+b] = [a'+b']$

$$(a+b) \equiv (a'+b') \pmod{n}$$

$$n \mid ((a+b) - (a'+b'))$$

$$= n \mid ((a-a') + (b-b'))$$

$$= n \mid (a-a') + n \mid (b-b')$$

$$(a+b) \equiv (a'+b') \pmod{n}$$

$$[a+b] = [a'+b']$$

2. Show $[ab] = [a'b']$

$$ab - a'b' = ab - a'b' + a'b' - a'b$$

$$= b(a-a') + a'(b-b')$$

$$= n \mid (a-a') \cdot n \mid (b-b')$$

$$= n \mid (ab - a'b')$$

$$ab \equiv a'b' \pmod{n}$$

$$[ab] = [a'b']$$

Check the ring axioms

1. $(\mathbb{Z}_n, +)$ is an abelian (additive) group

property	Proof
closure	$[a] + [b] = [a+b] \in \mathbb{Z}_n$
associate	$([a] + [b]) + [c] = [a] + ([b] + [c])$
commutative	$[a] + [b] = [a+b] = [b+a] = [b] + [a]$
identity	$[0]$; $[a] + [0] + [a+0] = [a]$
inverse	$[-a]$; $[a] + [-a] = [a-a] = [0]$

2. $(\mathbb{Z}_n, \cdot)$ is a commutative monoid

property	Proof
closure	$[a] [b] = [ab] \in \mathbb{Z}_n$
associate	$([a] [b]) \cdot [c] = [ab] \cdot [c] = [(ab)c] = [a(bc)] = [a] \cdot ([b] \cdot [c])$
commutative	$[a] \cdot [b] = [ab] = [ba] = [b] \cdot [a]$

distributivity of above properties

For all $[a], [b], [c] \in \mathbb{Z}_n$:

$$\begin{aligned}[a] \cdot ([b] + [c]) &= [a] [b+c] \\ &= [a(b+c)] \\ &= [ab + ac] \\ &= [ab] + [ac] \\ &= [a] \cdot [b] + [a] \cdot [c]\end{aligned}$$

As shown above, the
ring properties are satisfied.

Therefore, $(\mathbb{Z}_n, +, \cdot)$ is a ring.